AI : Report

TD1-WARGAME

Written by : Waneeza Hira AWAN

Group : 1

M1 SIC, 2026-2027.

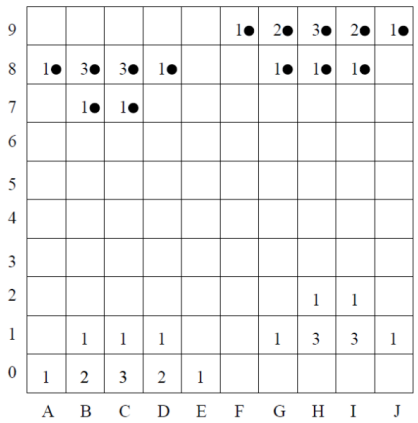


Table des matières

1. Introduction 2

1. Game Mechanics and Successor Generation

The game takes place on a 10x10 grid between two players whose goal is either to eliminate all enemy pieces or to breakthrough to the opponent's back rank. Each turn allows a single unit to move one square across eight possible directions, with battles occurring when entering an occupied square. Unlike standard chess where capture rules are straightforward, battles here are determined by local tactical support, meaning the sum of neighboring pieces around the target square. Before designing any evaluation or search algorithms, my primary technical motivation was to build a reliable mechanism to generate every possible transition from any board state, effectively providing the branching structure required by tree exploration.

Rather than copying entire board configurations at every prospective branch, which would quickly exhaust memory and slow down recursive execution, I chose to represent an action in its most compact form. I defined a lightweight Coup_structure that only records the initial and final coordinates (l1, c1) and (l2, c2). To aggregate these moves dynamically, I opted for a singly linked list (Liste_Coups). This choice was motivated by flexibility: since the number of legal moves varies significantly depending on the number of surviving pieces and their mobility, a linked list allocates only the exact amount of memory needed per ply. I complemented this structure with ajouter_coup for constant-time head insertion and liberer_coups to systematically release dynamic memory, preventing leaks across subsequent search depths.

The core branching logic is encapsulated within generer_coups. The function scans the entire grid to locate pieces belonging to the active player. Whenever a matching unit is found, it evaluates the eight neighboring squares by computing relative offsets and delegates rule compliance directly to the provided f_test_mouvement. Reusing this helper was an intentional architectural choice: it guarantees that boundary overflows, friendly fire, and invalid strides are handled consistently without redundant checks. Consequently, this first milestone establishes the foundation of the state-space search, providing valid child nodes ready to be explored by Minimax.